

Formal Methods for System Verification

Motivation, Background, Course Planning

Chandan Karfa



भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

Outline

- What is Verification?
- Why do we need to Verify?
- What do we verify?
- Why do we need Formal Methods for Verification?
- How Formal Verification Works?

What is Verification?

- **System verification** amounts to checking whether a system fulfills the specified **qualitative requirements**.
- It ensures that there are **no flaws in the implementation** and that the **implementation correctly meets the specification**.
- **Formal verification** is a mathematical proof method of ensuring that a design's implementation matches its specification.

Why do we need to Verify?

- Because Bugs exist in every software and hardware.
- CWE (Common Weakness Enumeration)
 - <https://cwe.mitre.org/>
- Someone is careless during implementation.
 - <https://www.cs.tau.ac.il/~nachumd/horror.html>

- Knowing the weaknesses that result in vulnerabilities means software developers, hardware designers, and security architects can eliminate them before deployment, when it is much easier and cheaper to do so.

699 - Software Development

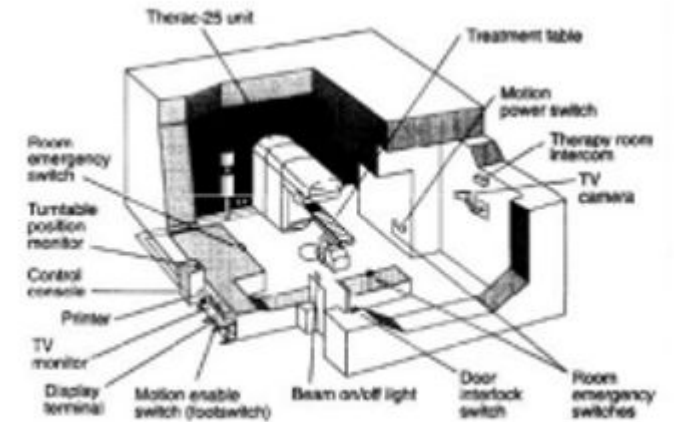
- ☐ ☒ API / Function Errors - (1228)
- ☐ ☒ Audit / Logging Errors - (1210)
- ☐ ☒ Authentication Errors - (1211)
- ☐ ☒ Authorization Errors - (1212)
- ☐ ☒ Bad Coding Practices - (1006)
- ☐ ☒ Behavioral Problems - (438)
- ☐ ☒ Business Logic Errors - (840)
- ☐ ☒ Communication Channel Errors - (417)
- ☐ ☒ Complexity Issues - (1226)
- ☐ ☒ Concurrency Issues - (557)
- ☐ ☒ Credentials Management Errors - (255)
- ☐ ☒ Cryptographic Issues - (310)
- ☐ ☒ Key Management Errors - (320)
- ☐ ☒ Data Integrity Issues - (1214)
- ☐ ☒ Data Processing Errors - (19)
- ☐ ☒ Data Neutralization Issues - (137)
- ☐ ☒ Documentation Issues - (1225)
- ☐ ☒ File Handling Issues - (1219)
- ☐ ☒ Encapsulation Issues - (1227)
- ☐ ☒ Error Conditions, Return Values, Status Codes - (389)
- ☐ ☒ Expression Issues - (569)

1194 - Hardware Design

- ☐ ☒ Manufacturing and Life Cycle Management Concerns - (1195)
- ☐ ☒ Security Flow Issues - (1196)
- ☐ ☒ Integration Issues - (1197)
- ☐ ☒ Privilege Separation and Access Control Issues - (1198)
- ☐ ☒ General Circuit and Logic Design Concerns - (1199)
- ☐ ☒ Core and Compute Issues - (1201)
- ☐ ☒ Memory and Storage Issues - (1202)
- ☐ ☒ Peripherals, On-chip Fabric, and Interface/IO Problems - (1203)
- ☐ ☒ Security Primitives and Cryptography Issues - (1205)
- ☐ ☒ Power, Clock, Thermal, and Reset Concerns - (1206)
- ☐ ☒ Debug and Test Problems - (1207)
- ☐ ☒ Cross-Cutting Problems - (1208)
- ☐ ☒ Physical Access Issues and Concerns - (1388)

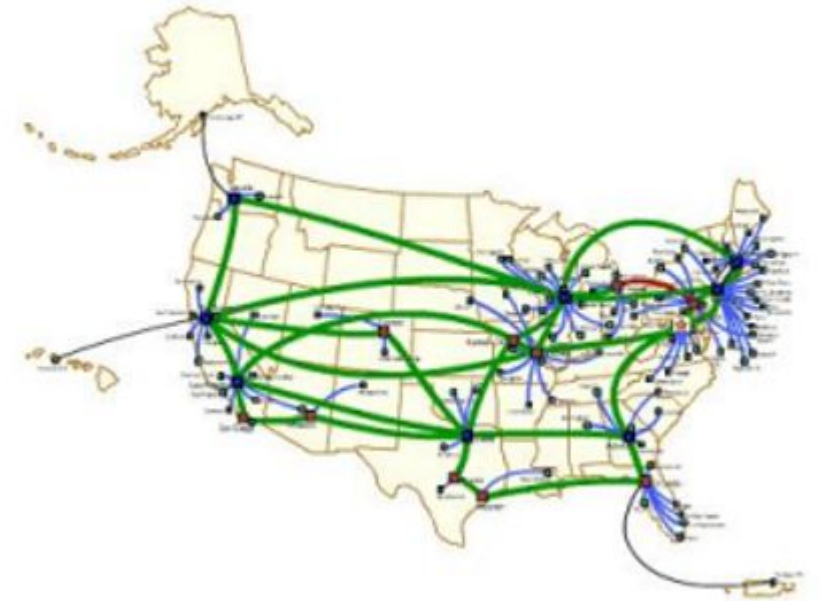
Therac-25 Radiation Overdosing (1985–87)

- Radiation machine for treatment of cancer patients
- At least 6 cases of overdose in period 1985–1987 ($\approx$ 100-times dosage)
- Three cancer patients died
- Source: Design error in the control software (race condition)
- Software written in assembly language



AT&T Telephone Network Outage (1990)

- A problem in New York City led to a 9-hour outage affecting large parts of the U.S. telephone network
- Costs: Several hundred million US dollars
- Source: Software flaw(wrong interpretation of the break statement in C)



Ariane 5 Crash (1996)

- Crash of the European Ariane 5 missile in June 1996
- Costs: More than 500 million US dollars
- Source: [Software flaw in the control software](#)
- Cause: A data conversion from a 64-bit floating-point value to a 16-bit signed integer



Pentium FDIV Bug (1994)

- FDIV = Floating-point division unit
- Certain floating-point division operations produced incorrect results
- Rate: Approximately 1 in 9 billion floating-point divisions with random inputs would yield an inaccurate result
- Loss: $\approx$ 500 million US dollars
(all flawed processors were replaced) + significant reputational damage to Intel
- Source: [Incorrect realization of the floating-point division algorithm](#)



Some Recent Bugs

- **Google Tensor SoC Thermal Throttling Issue (2023)**
- **Type:** Power / thermal management bug
Detected by: Post-silicon validation
- Unexpected throttling under sustained AI workloads.
- Thermal control logic was functionally correct but **sub-optimal**.
- *Verification lesson:* Functional correctness $\neq$ performance correctness.

Boeing 787 Power Control Logic Bug

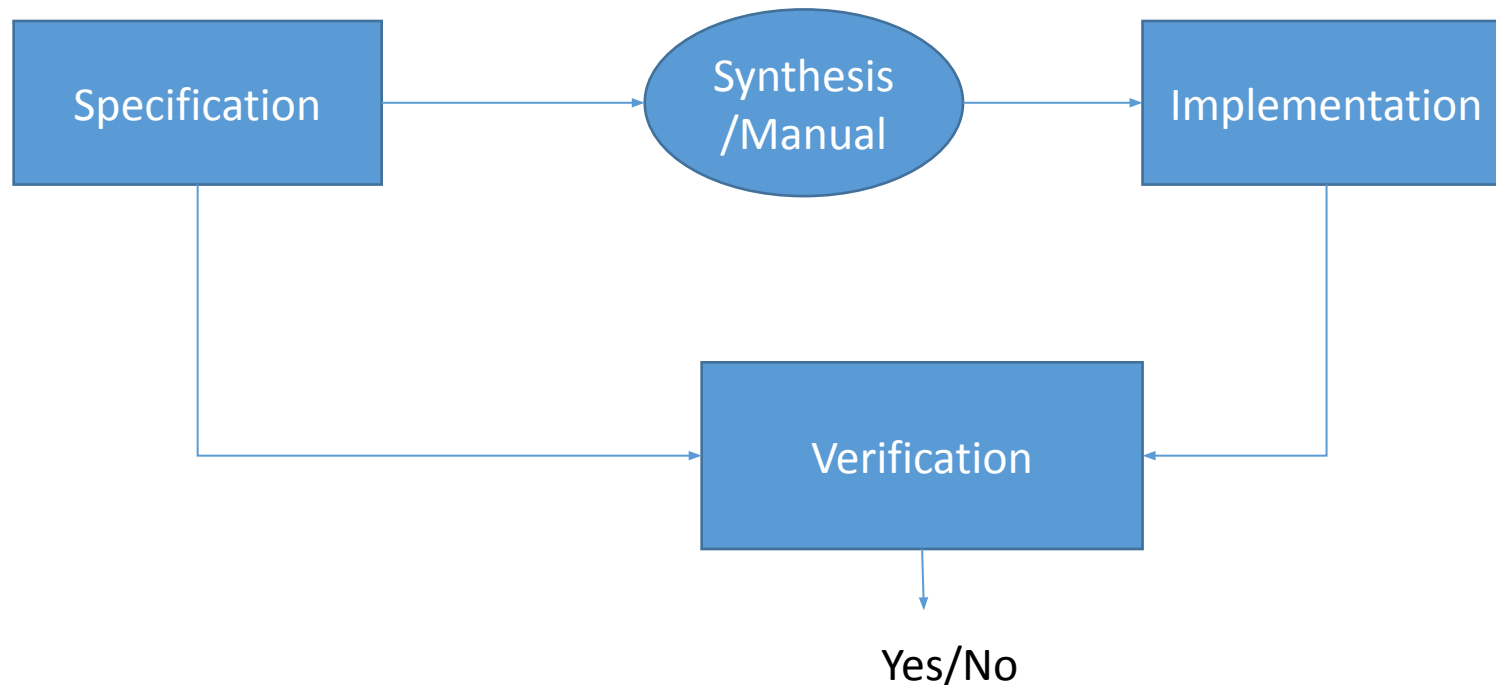
- **Boeing 787 Power Control Logic Bug (Reported 2023)**
- **Type:** Embedded system logic bug
Detected by: System validation
- A power-control unit bug could disable generators after continuous operation.
- Root cause: **counter overflow / reset logic.**
- *Verification lesson:* Boundary conditions and long-uptime scenarios matter.

Apple M-Series Speculation Bug (2024)

- **Type:** Micro-architectural security bug
Detected by: Academic researchers
- Data leakage via speculative execution on Apple Silicon.
- Bypassed OS-level protections.
- No functional failure, only security exposure.
- *Verification lesson:* Formal methods needed for micro-architectural security.

What do we Verify?

- The implementation correctly implements the specification.
- Is my implementation correct?



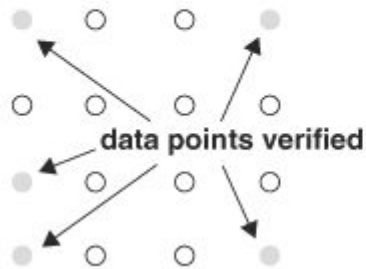
How do we verify?

- Simulation based Verification
 - Based on set of inputs
- Formal verification
 - Mathematically without any inputs

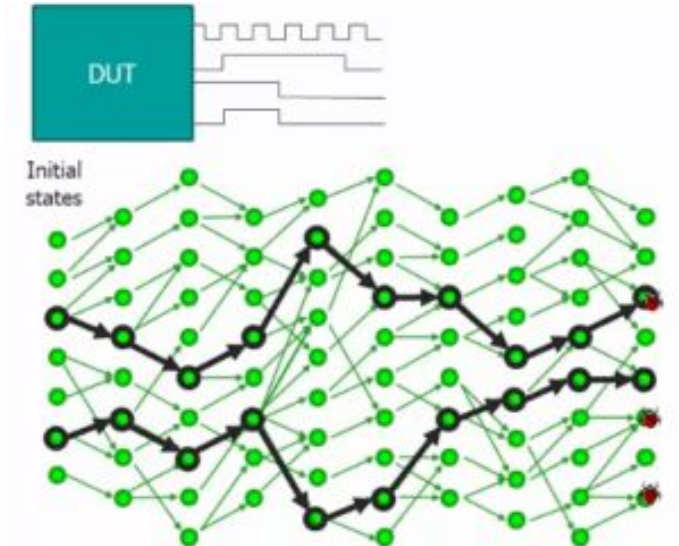
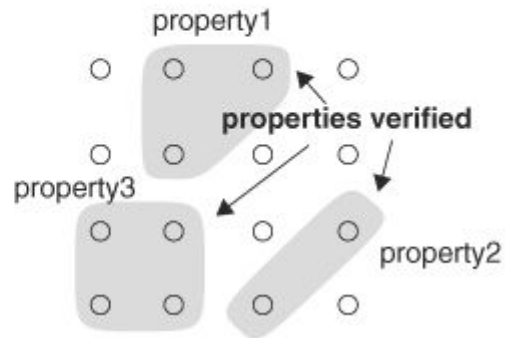
Why do we need Formal Methods for Verification?

- Simulation based verification vs Formal Verification

simulation-based verification



formal verification



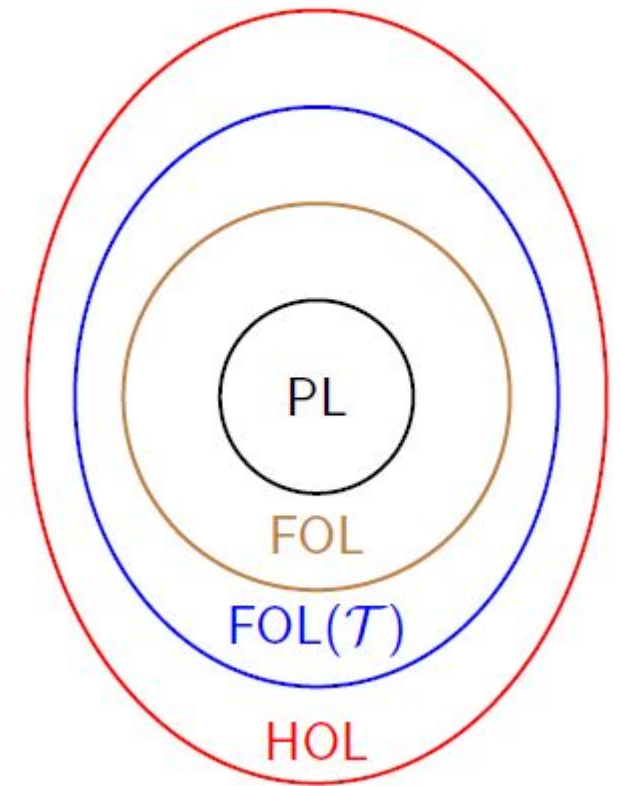
How do we formally Verify?

Applies mathematics for modelling and Analyzing Systems

- Theorem Proving
- Model Checking
- Equivalence Checking

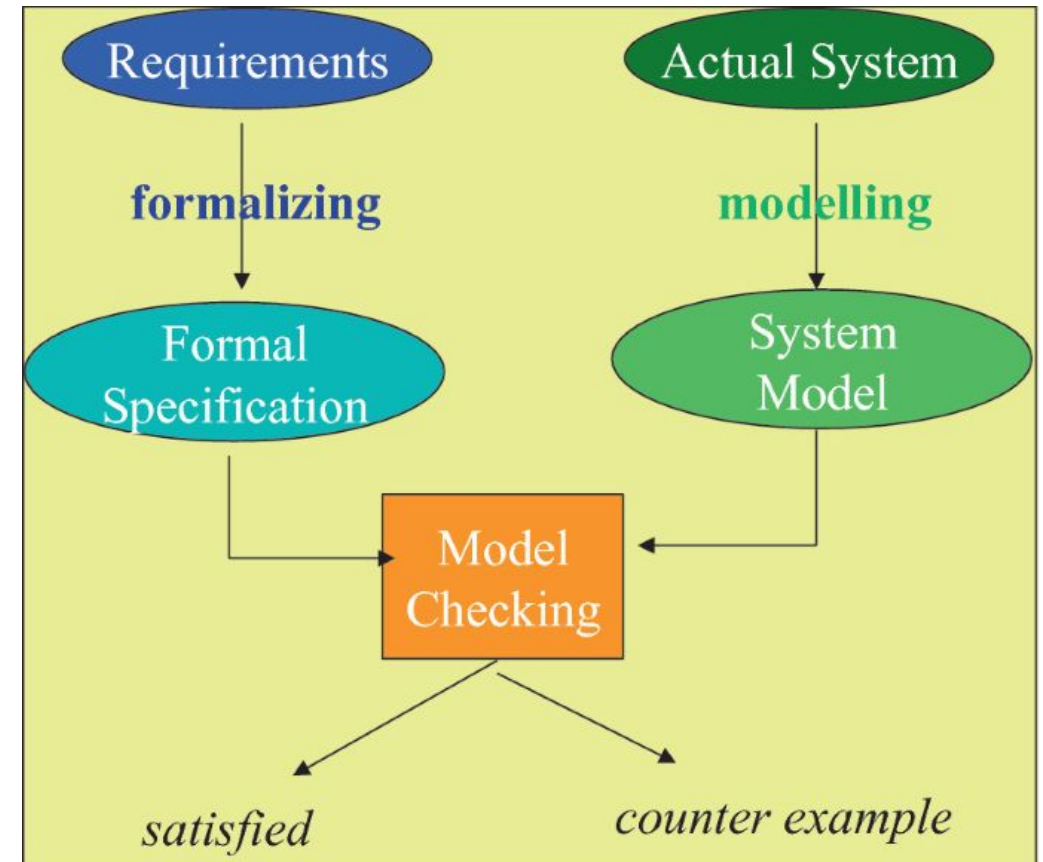
Theorem Proving

- Set of Axioms (A)
- A formula that captures the implementations
- Is the formula satisfiable under the axioms?
- Spectrum of Logic
 - Propositional Logic (PL)
 - First order logic (FOL)
 - First order logic with Theories (FOL(T))
 - Higher order logic (HOL)
- Solvers
 - SAT solvers
 - SMT Solvers



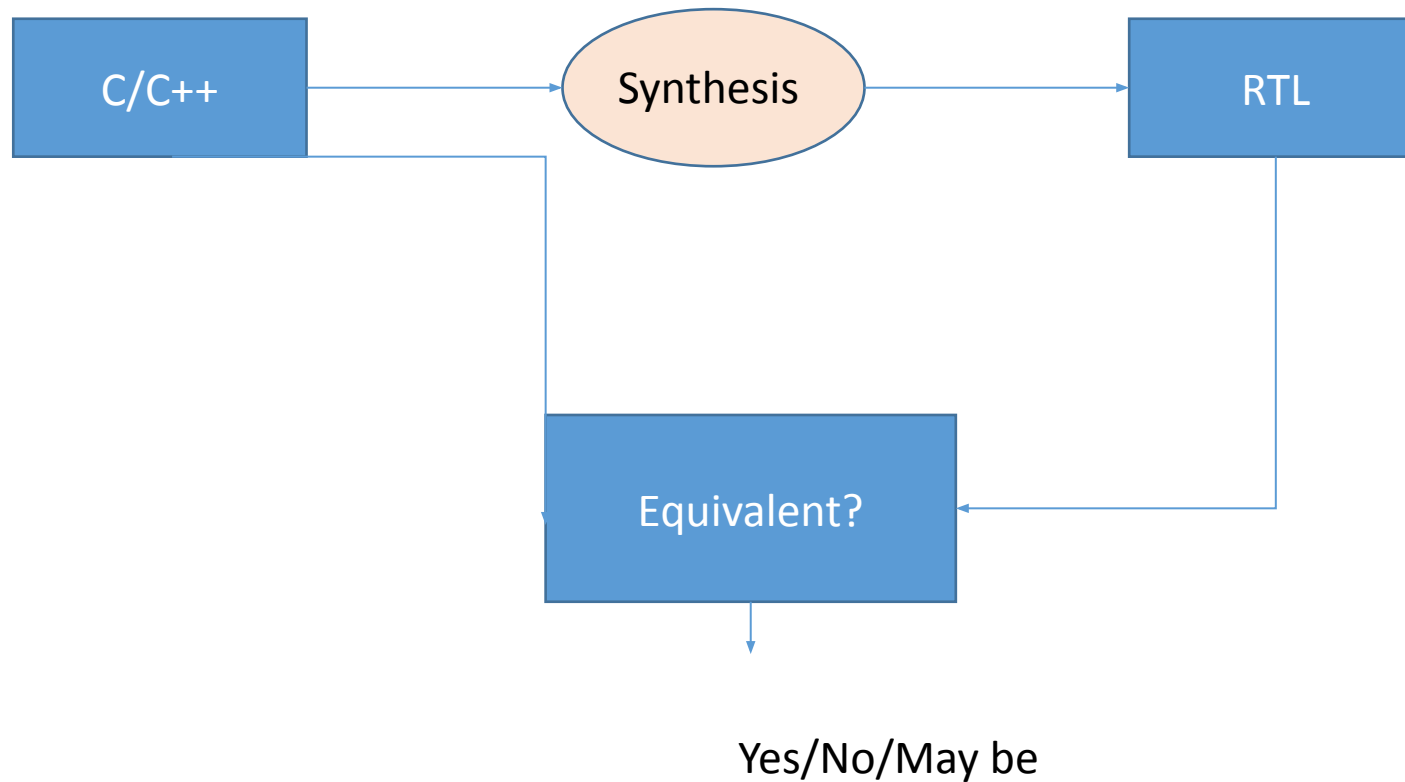
Model Checking

- Temporal Logic
 - LTL
 - CTL
 - Model checking algorithm



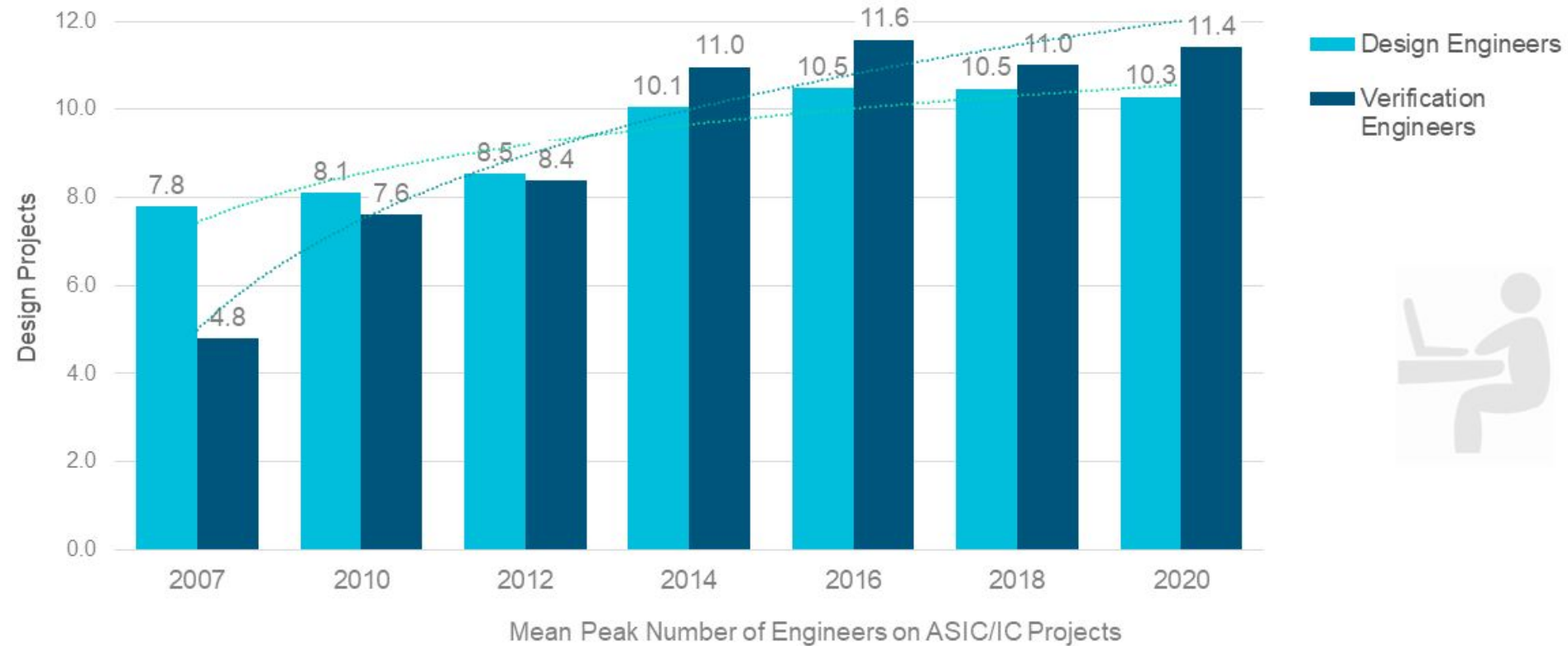
Equivalence Checking

- Formal model
- Algorithm
- Correctness proofs

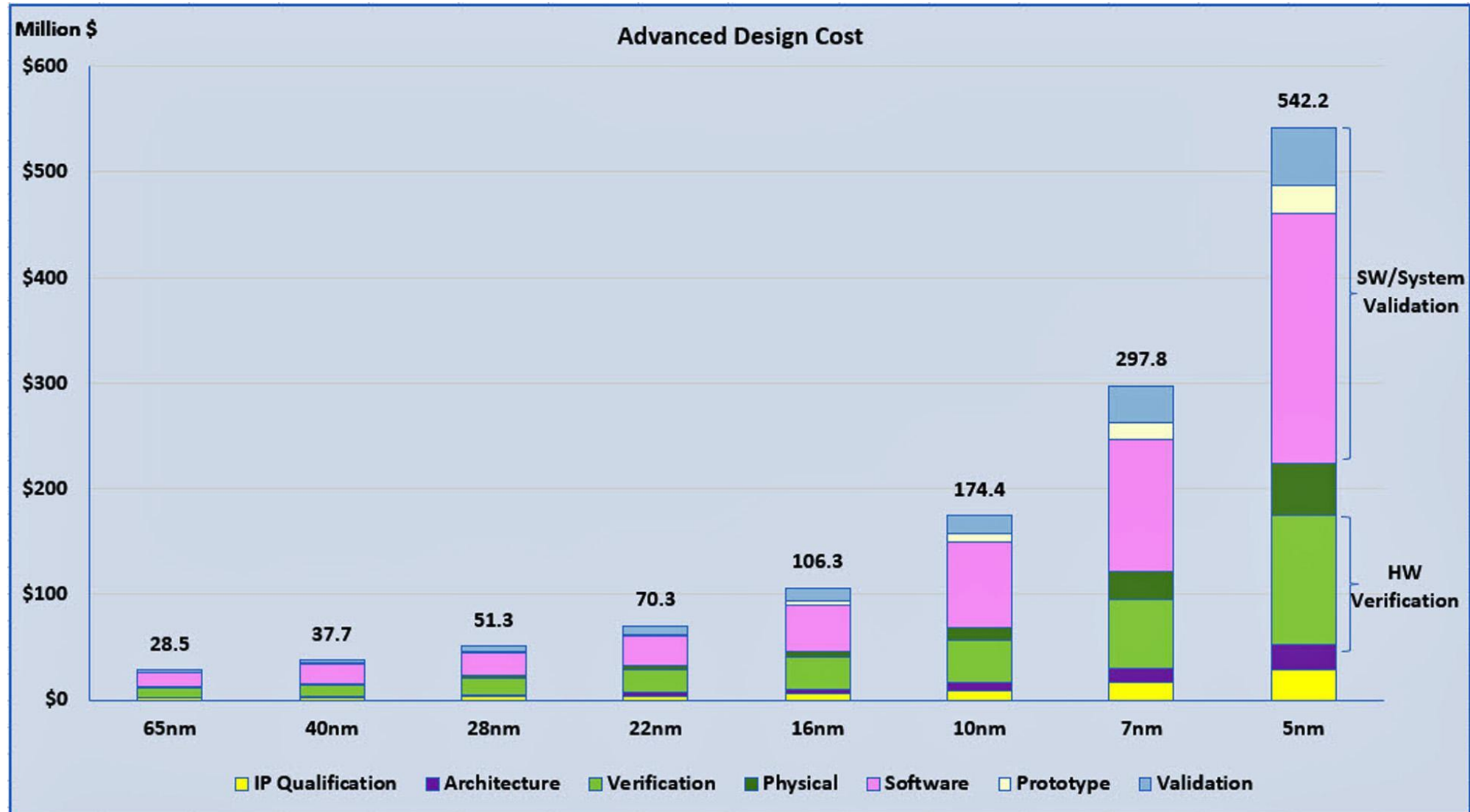


Design Efforts

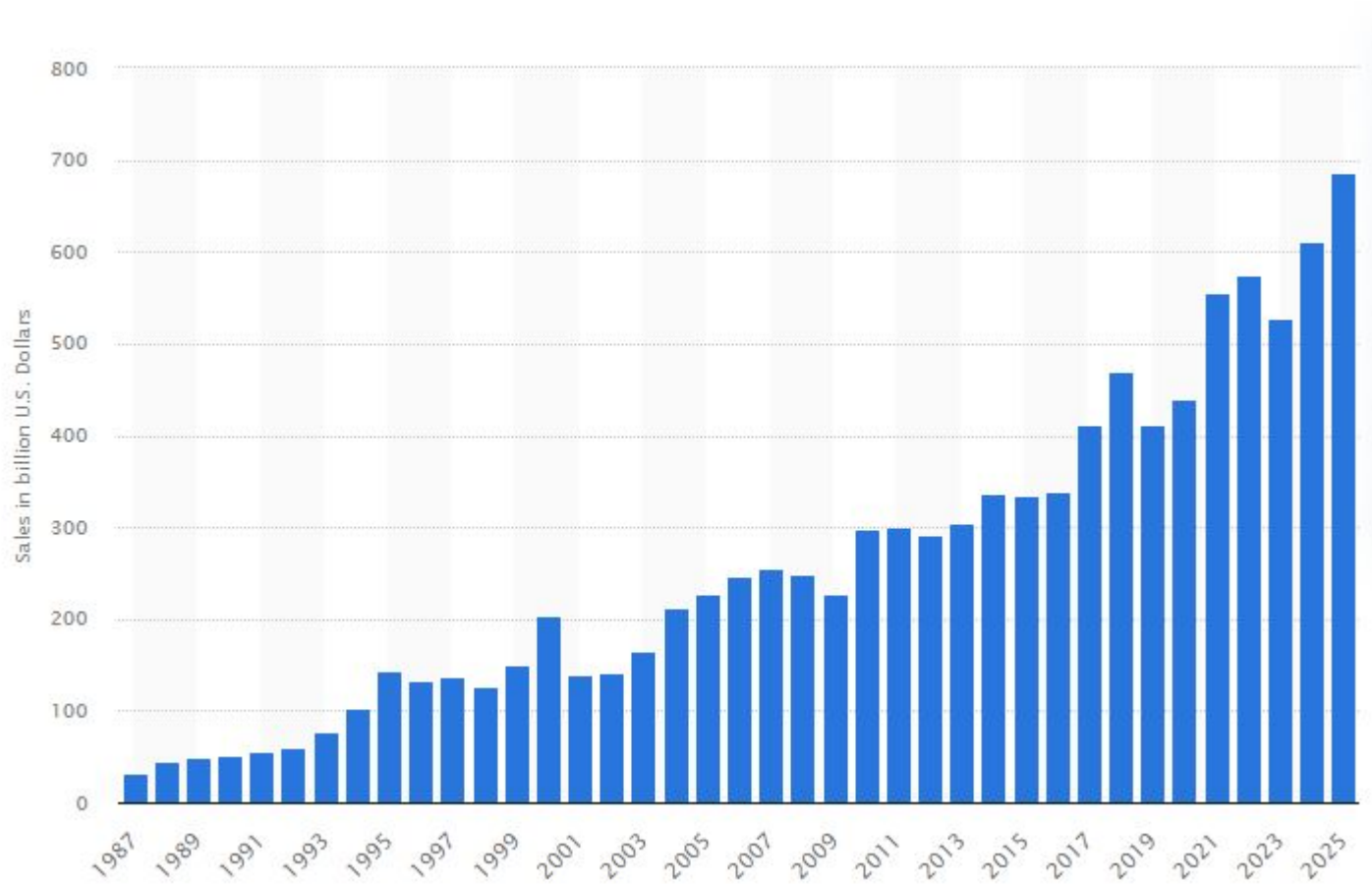
Mean Peak Number of Engineers on an ASIC/IC Project



Design Cost

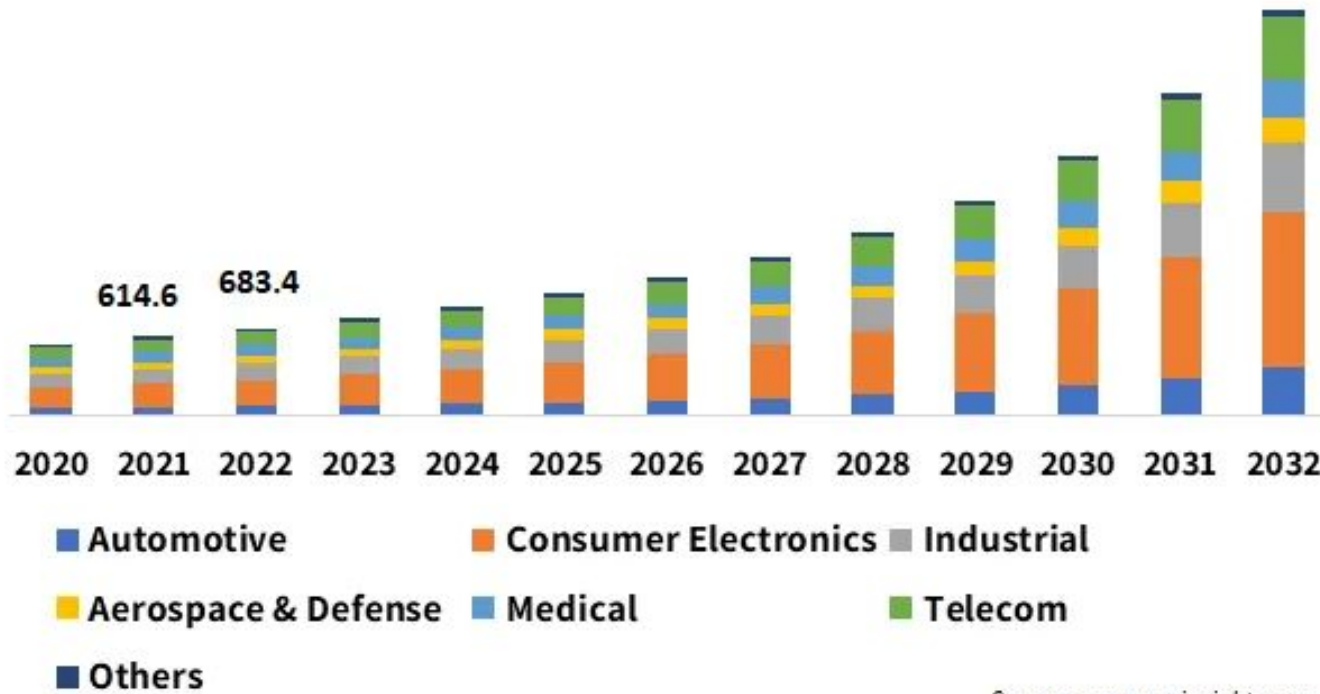


Semiconductor market revenue



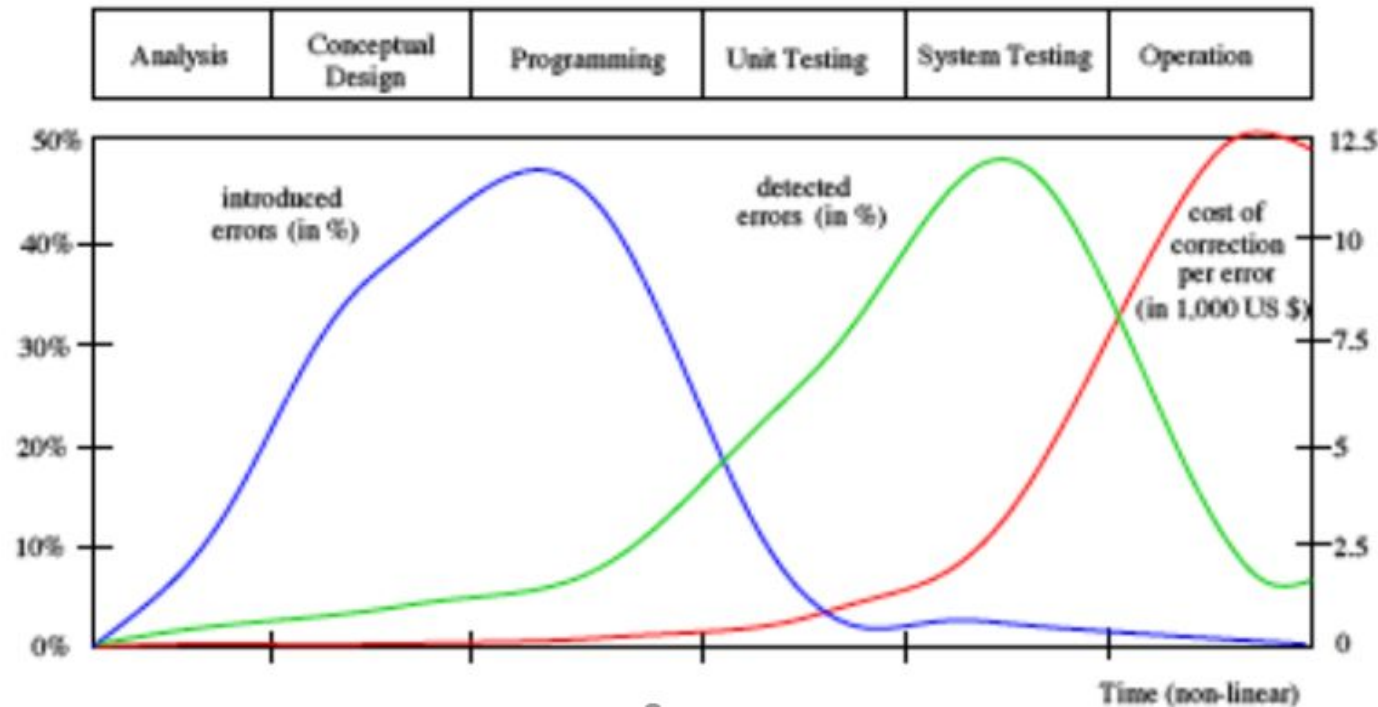
Verification Market

Global Hardware Assisted Verification Market, 2020-2032 (USD Million)



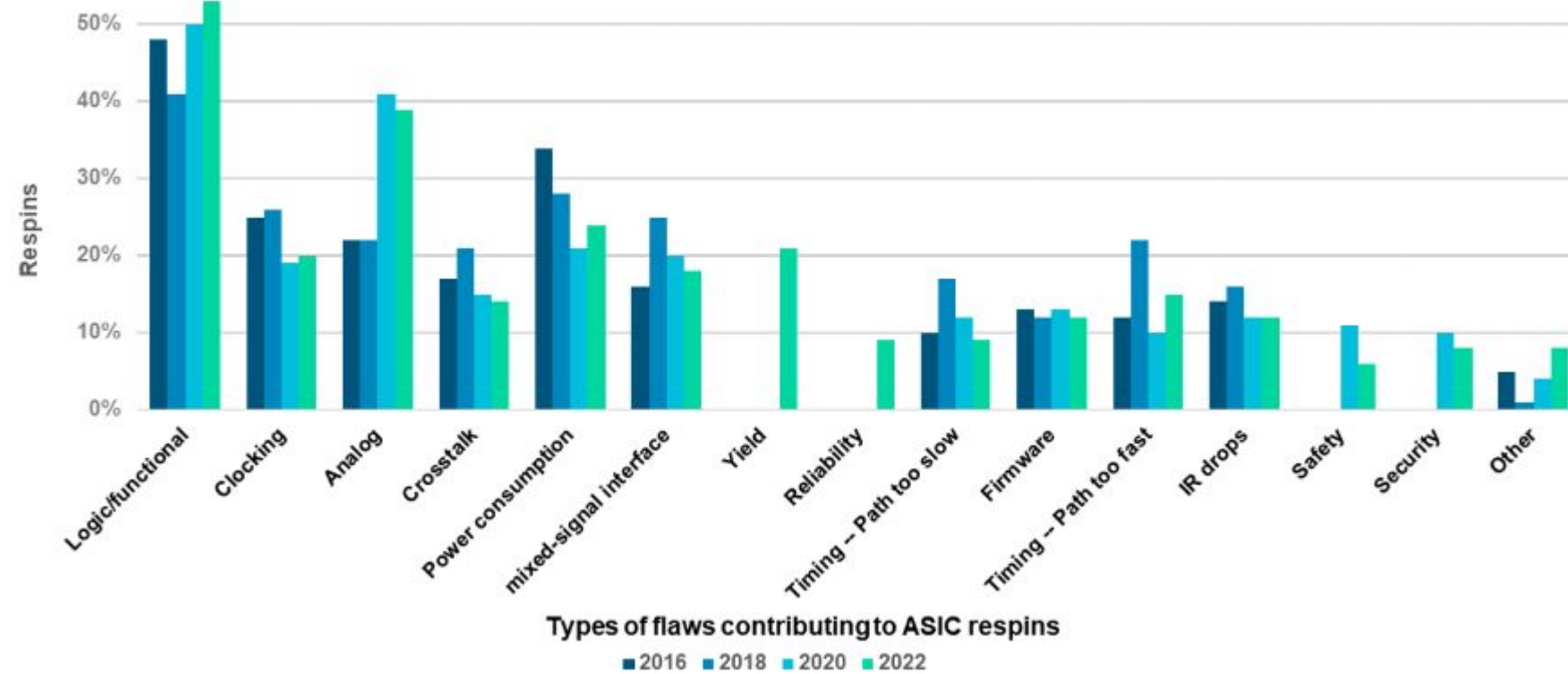
Source: www.gminsights.com

The Sooner, the Better



Cause of ASIC respins

Logic/functional failures consistently the top cause of respins



Source: Wilson Research Group and Siemens EDA, 2022 Functional Verification Study

Unrestricted | © Siemens 2022 | Functional Verification Study

* Multiple replies possible

Course Content

- Introduction to formal methods and hardware verification.
- Review of logics:
 - Propositional Calculus and Predicate Calculus.
 - Describing hardware directly in higher order logic.
 - Combinational and sequential behaviour of circuits.
- Specification of hardware systems.
 - Introduction to Binary Decision Diagram (BDD) and modelling hardware with BDDs.
 - Algorithms for BDD operations.
 - Concept of OBDDs and ROBDDs and operation on ROBDDs.
- SAT Solver
- SMT Solver

Course Content

- Formal Property Verification
 - Introduction to Temporal Logic.
 - Linear and Branching time temporal logic.
 - Expressing properties in CTL and LTL.
 - CTL and LTL model checking algorithms.
 - State space explosion problem: Symbolic data structure and symbolic model checking algorithms.
 - Bounded Model Checking
- LLM/AI aided verification
- Case Studies

Reference Books

- Daniel Kroening and Ofer Strichman: Decision Procedures - An Algorithmic Point of View, Springer, 2008
- M. Huth and M. Ryan, Logic in Computer Science: Modelling and Reasoning about Systems, 2nd Ed, Cambridge University Press, 2004.
- Principles of Model Checking, Christel Baier and Joost-Pieter Katoen, MIT Press, May 2008.
- Aaron R. Bradley, Zohar Manna: The Calculus of Computation, Springer 2010.

ACM Turing Award 2007



Edmund Clarke



E. Allen Emerson



Joseph Sifakis

"For their role in developing Model-Checking into a highly effective verification technology, widely adopted in the hardware and software industries."

*Thank
you!*